

Amazones – Python

Rapport final

Messaoudi Ismail, Mohammed Amine Rabih, Mohamed Ali Chaoui, Ayoub Oubakki, Mahmoud Mounouar

10 avril 2026

Table des matières

1	Introduction	3
1.1	Jeu et travaux existants	3
1.2	Contexte et objectif du projet	3
1.3	Complexité du jeu	3
1.4	Exigences du sujet	3
2	Modélisation du jeu	3
2.1	Plateau, état de jeu et structure d'un tour	3
2.2	Placement adaptatif selon la taille du plateau	4
2.3	Notation algébrique (F20)	4
3	Structures de données	5
3.1	Bitboards	5
3.1.1	Indexation des cases	5
3.1.2	Opérations bit à bit	5
3.1.3	Visualisation des opérations critiques	6
4	Algorithmes	6
4.1	Génération et validation des coups	6
4.1.1	Problème du wrapping	7
4.1.2	Algorithme complet	7
4.2	Algorithmes d'intelligence artificielle	7
4.2.1	Minimax avec élagage $\alpha\beta$	7
4.2.2	Fonctions d'évaluation	8
4.2.3	Monte Carlo Tree Search (MCTS)	8
4.2.4	Comparaison critique des approches	8
5	Architecture du système	8
5.1	Architecture générale	9
5.1.1	Pattern MVC	9
5.1.2	Découpage en modules	10
5.1.3	Module Model	11
5.1.4	Module Controller	11
5.1.5	Module View	12
5.1.6	Interfaces principales entre modules	12
5.1.7	Diagrammes de séquence	13
5.1.8	Architecture réseau	14
5.1.9	Intégration des fonctionnalités avancées	15
5.1.10	Internationalisation	15
5.2	Préoccupation de qualité logicielle	15

6	Explication approfondie	16
6.1	Fonctionnalité 1 — Génération des coups avec bitboards	16
6.2	Fonctionnalité 2 — IA Minimax avec élagage $\alpha\beta$	17
7	Implémentation	17
7.1	Moteur de jeu et structures internes	17
7.1.1	Représentation mémoire	17
7.1.2	Génération des coups légaux — Implémentation détaillée	18
7.1.3	Application et annulation de coups	18
7.1.4	Validation des coordonnées	18
7.2	Implémentation de l'intelligence artificielle	18
7.2.1	Architecture de l'IA	18
7.2.2	Sélection du moteur	18
7.2.3	Implémentation Minimax avec élagage $\alpha\beta$	18
7.2.4	Implémentation MCTS	18
7.2.5	Approfondissement itératif et budget temps	19
7.2.6	Évaluateur territorial (BFS)	19
7.2.7	Gestion du temps	19
7.3	Persistence, interfaces et intégration	19
7.3.1	Sauvegarde et chargement des parties	19
7.3.2	Intégration de la GUI	19
7.3.3	Intégration du réseau	20
8	Tests et validation	20
8.1	Stratégie de test	20
8.2	Outils et critères de validation	21
9	Analyse de performances	21
9.1	Mesures	21
9.2	Comparaison expérimentale des IA	21
9.3	Interprétation	22
10	Discussion critique	22
10.1	Limites algorithmiques	22
10.2	Limites architecturales	23
10.3	Limites système	23
10.4	Améliorations possibles	23
11	Apports du projet	23
11.1	Apports techniques	23
11.2	Apports méthodologiques	23
11.3	Vision d'ingénieur	23
	Conclusion	23
A	Liste des 40 besoins fonctionnels	25

1 Introduction

1.1 Jeu et travaux existants

Le jeu des Amazones existe depuis 1988 et a fait l'objet de nombreux travaux sur la recherche adversariale, l'évaluation territoriale et les politiques de simulation. Les travaux de Müller [8], les analyses de Lieberum [7] et les compétitions de *Computer Olympiads* montrent tous la même difficulté : le jeu combine un facteur de branchement élevé, une forte dimension spatiale et une fragmentation progressive du plateau.

Ces références servent surtout à situer le sujet. Les Amazones sont un jeu déterministe à information parfaite, difficile à résoudre, et assez riche pour imposer de vrais choix de structures de données et d'algorithmes.

1.2 Contexte et objectif du projet

Le projet *Amazones – Python* regroupe un moteur de jeu, une CLI, une GUI, des IA, une sauvegarde et un mode réseau. La difficulté est de faire reposer ces usages sur un même noyau de règles.

Du point de vue théorique, Hearn [4] a montré que le problème de décision associé au jeu est **PSPACE-complet**. Dans notre cas, cela se traduit par trois besoins concrets : représenter le plateau à faible coût, générer beaucoup de coups sans perdre en cohérence, et produire une décision dans un temps borné.

Ce rapport explique comment ces contraintes ont été traduites dans le code, au niveau des structures de données, des algorithmes et de l'architecture.

1.3 Complexité du jeu

Les Amazones sont un jeu déterministe à information parfaite. Leur difficulté ne vient donc ni du hasard ni d'une information cachée, mais du nombre de décisions possibles à chaque tour et de l'effet durable de chaque flèche sur l'espace de jeu. Un coup complet combine en effet un déplacement de reine puis un tir de flèche, ce qui multiplie rapidement le nombre de successeurs possibles.

Sur un plateau standard 10×10 , le nombre de coups légaux devient très élevé dès le début de partie. À mesure que le plateau se ferme, la lecture stratégique se déplace vers le contrôle de zones et l'isolement progressif des amazones. Cette difficulté explique directement le recours à des représentations compactes et à des moteurs de décision bornés par le temps.

1.4 Exigences du sujet

Le sujet imposait un logiciel complet, et non un simple prototype de démonstration. Les exigences structurantes peuvent se résumer ainsi :

- implémenter un noyau de jeu correct, avec validation, historique, undo/redo et notation algébrique ;
- supporter plusieurs modes d'utilisation : CLI, GUI, jeu contre l'IA et mode réseau ;
- fournir des mécanismes de persistance, des tests explicites et une organisation du code suffisamment claire pour être maintenue.

La liste détaillée des quarante besoins est placée en annexe. Dans le corps du rapport, nous gardons seulement les choix de modélisation, de structures de données, d'algorithmes et d'architecture qui expliquent vraiment le fonctionnement du logiciel.

2 Modélisation du jeu

Cette section formalise l'état du jeu, la structure d'un tour et la notation utilisée dans la suite du document. Elle fixe le vocabulaire et les conventions qui seront réutilisés dans les sections techniques.

2.1 Plateau, état de jeu et structure d'un tour

Le jeu se déroule sur un plateau carré de taille $N \times N$ (configurable de 4×4 à 11×11 , standard : 10×10). Chaque joueur contrôle un ensemble d'amazones, au nombre de quatre dans la configuration classique. Une position peut être décrite formellement par l'état

$$s = (B_W, B_B, B_X, p, h, \tau),$$

où B_W et B_B représentent les positions des amazones blanches et noires, B_X l'ensemble des cases bloquées par des flèches, p le joueur courant, h l'historique des coups et τ les informations de chronométrage. Cette écriture rappelle qu'une partie ne se réduit pas à un plateau : elle inclut aussi un joueur courant, un historique exploitable par l'annulation et, dans certains modes, un état de temps.

Un coup complet est une action composée de deux étapes indissociables :

1. **Déplacement** : le joueur actif choisit une amazone et la déplace comme une reine aux échecs, c'est-à-dire horizontalement, verticalement ou en diagonale, sur un nombre quelconque de cases libres.
2. **Tir de flèche** : depuis sa nouvelle position, cette amazone tire une flèche suivant les mêmes règles de déplacement ; la case atteinte est alors bloquée de manière irréversible.

Dans l'interface graphique, chaque amazone est représentée par une icône de reine. Ce choix est volontaire : il rappelle immédiatement que la pièce se déplace comme la reine aux échecs avant le tir de flèche. Autrement dit, si l'on note $a = (m, f)$ un coup composé d'un mouvement m puis d'un tir f , la transition d'état $\delta(s, a)$ n'est valide que si les deux sous-actions satisfont les conditions géométriques et de collision du plateau.

La fin de partie survient lorsqu'un joueur ne possède plus aucun coup légal. Le jeu est donc **déterministe**, à **information parfaite** et **terminant** : à chaque tour, une case libre est détruite par un tir de flèche, ce qui fait décroître monotoniquement l'espace de jeu. Il n'existe ni capture ni hasard. Cette absence de capture simplifie certaines règles, mais elle déplace la difficulté stratégique vers le contrôle de l'espace et l'anticipation des zones de blocage.

Ces propriétés fixent le cadre du moteur. Chaque décision transforme durablement l'espace de jeu.

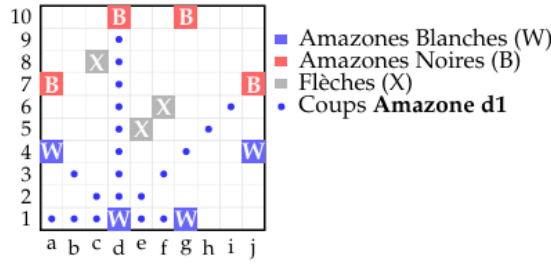


FIGURE 2.1 – Vue synthétique du plateau avec coordonnées algébriques, amazones blanches et noires, flèches bloquantes et coups légaux illustrés pour l'amazone blanche sélectionnée.

La Figure 2.1 sert de référence visuelle pour la suite. Elle montre les coordonnées du plateau, la distinction entre amazones blanches (W), amazones noires (B) et flèches (X), ainsi qu'un exemple de coups légaux déjà calculés pour une amazone donnée. Les coordonnées utilisées dans les exemples textuels doivent être lues sur ce même repère.

2.2 Placement adaptatif selon la taille du plateau

La taille du plateau étant paramétrable, un placement unique des amazones n'aurait pas été satisfaisant. Le projet retient donc une stratégie adaptative dont l'objectif est de préserver une symétrie minimale et une jouabilité acceptable sur plusieurs dimensions :

Taille ($N \times N$)	Nb Amazones	Stratégie
4×4	2	Coin opposés
5×5	3	Triangle symétrique
6×6 à 11×11	4	Formule des Tiers

TABLE 2.1 – Stratégies de placement selon la taille du plateau

Pour $N \geq 6$, les positions sont calculées par l'algorithme des Tiers : $T = \lfloor N/3 \rfloor$, avec les amazones blanches placées aux positions $(0, T)$, $(0, N-1-T)$, $(T, 0)$ et $(T, N-1)$, et les noires par symétrie centrale. Ce choix maintient une distribution équilibrée sur les bords sans figer le moteur sur la seule configuration classique 10×10 .

2.3 Notation algébrique (F20)

Les cases sont identifiées par une colonne (lettre a à j) et une ligne (chiffre 1 à 10). Les coups suivent la notation algébrique standard :

$e2-e4/e6 \rightarrow$ déplacer l'amazone de e2 vers e4, puis tirer une flèche sur e6.

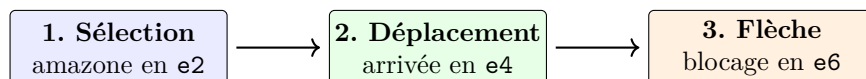


FIGURE 2.2 – Exemple concret d'un coup complet : $e2-e4/e6$.

Cette notation sera réutilisée dans l'ensemble du projet. Dans le rapport, elle relie directement la description du coup, la Figure 2.1 et l'exemple de la Figure 2.2.

3 Structures de données

Cette section présente la représentation interne retenue pour manipuler le plateau. Le choix central est celui des bitboards : il répond au coût élevé des tests d'occupation, des mises à jour locales et de la génération répétée de coups.

3.1 Bitboards

Conformément aux exigences F26 et F27, le plateau est représenté par des **bitboards** [1]. Un bitboard est un entier de n^2 bits où chaque bit correspond à une case du plateau. L'état du jeu est encodé par **trois bitboards distincts** :

Bitboard	Contenu
white_bb	Positions des amazones blanches
black_bb	Positions des amazones noires
arrow_bb	Positions des flèches (cases bloquées)

TABLE 3.1 – Les trois bitboards représentant l'état du jeu

Une matrice $N \times N$ aurait été plus intuitive, mais elle n'était pas la plus adaptée ici. Les bitboards ont été retenus pour trois raisons :

- ils concentrent l'état du plateau dans quelques entiers ;
- ils rendent les tests d'occupation et les mises à jour très peu coûteux ;
- ils s'intègrent directement aux besoins du moteur et des IA, qui manipulent un grand nombre d'états successifs sans conversion intermédiaire vers une matrice ou une structure de graphe.

Le coût du choix est connu : l'indexation devient plus abstraite et le code moins immédiat à lire qu'avec une matrice. Ce compromis a été accepté parce que, dans ce projet, les opérations critiques ne lisent presque jamais le plateau pour l'afficher ; elles testent surtout des occupations, construisent des états temporaires et reviennent en arrière très souvent.

3.1.1 Indexation des cases

Chaque case possède un index unique : $\text{index} = \text{ligne} \times n + \text{colonne}$, avec ligne et colonne $\in [0, n-1]$.

Case (affichée)	Coord. interne	Calcul ($n = 10$)	Index
a1	(9, 0)	$9 \times 10 + 0$	90
d4	(6, 3)	$6 \times 10 + 3$	63
j10	(0, 9)	$0 \times 10 + 9$	9

TABLE 3.2 – Correspondance notation algébrique $\leftrightarrow$ index bitboard (plateau 10×10)

Note : l'indexation interne commence en haut à gauche, alors que la notation algébrique place **a1** en bas à gauche. La conversion **algèbre** $\rightarrow$ **index** doit donc inverser l'axe vertical, ce qui explique que **a1** ne corresponde pas à l'index 0 dans l'implémentation réelle.

3.1.2 Opérations bit à bit

Les opérations fondamentales permettent de manipuler l'état du plateau en temps constant $O(1)$:

Opération	Notation	Usage
Shift gauche	$1 \ll n$	Créer un masque pour la position n
OR	$a \mid b$	Placer une pièce ou bloquer une case
XOR	$a \oplus b$	Retirer une pièce (inverser un bit)
AND	$a \wedge b$	Tester si une case est occupée

TABLE 3.3 – Récapitulatif des opérations bit à bit

Application d'un coup (ordre strict) :

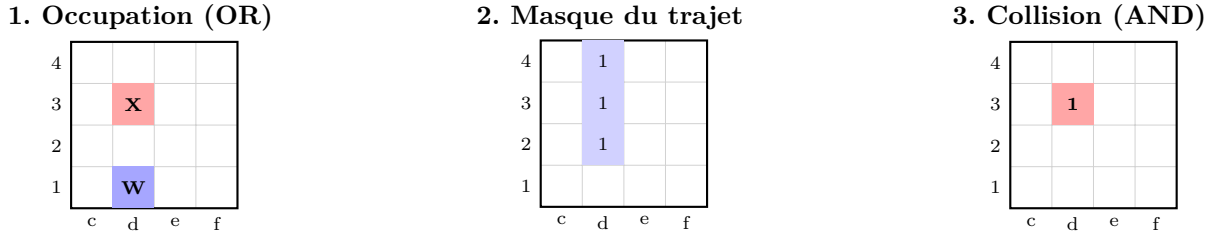
1. **Libérer** la case de départ : $\text{amazons} = \text{amazons} \oplus (1 \ll \text{from})$
2. **Occuper** la case d'arrivée : $\text{amazons} = \text{amazons} \mid (1 \ll \text{to})$
3. **Tirer la flèche** : $\text{arrows} = \text{arrows} \mid (1 \ll \text{arrow})$

Note : L'ordre est crucial car la flèche peut légalement atterrir sur la case **from** libérée à l'étape 1.

3.1.3 Visualisation des opérations critiques

Les figures suivantes détaillent deux opérations centrales du moteur bitboard : la détection d'une collision sur un chemin de déplacement, puis l'écriture effective d'un déplacement dans les structures binaires.

Validation d'un coup (lecture) Pour vérifier si un coup est valide, le moteur agrège d'abord les obstacles (OR), construit ensuite un masque correspondant au chemin voulu, puis teste l'intersection via AND. Toute intersection non nulle signale une collision.



Le masque de déplacement sur la colonne **d** rencontre une case déjà occupée en **d3** ; l'intersection est donc non nulle et le coup est refusé.

FIGURE 3.1 – Validation d'un coup : l'opération AND détecte la collision sur un plateau 10×10 .

La validité d'un déplacement est donc ramenée à une suite d'opérations binaires sur des masques de trajet et d'occupation, et non à un parcours matriciel case par case.

Application d'un coup (écriture) Le déplacement ne se fait pas par copie de plateau, mais par modification ciblée de bits : on retire d'abord l'amazone de sa case de départ par XOR, puis on active la case d'arrivée par OR.

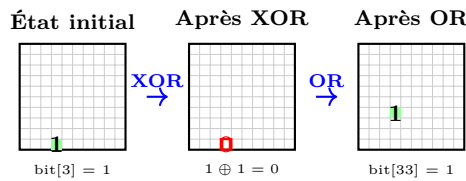


FIGURE 3.2 – Déplacement d'une amazone : XOR retire, OR place (plateau 10×10).

Cette représentation montre l'intérêt pratique des bitboards : un déplacement complet se traduit par quelques opérations élémentaires sur des entiers, sans recopier tout le plateau. Cet avantage devient déterminant lorsqu'une IA doit tester un grand nombre de coups successifs.

4 Algorithmes

Cette section présente d'abord la génération et la validation des coups, puis les moteurs d'intelligence artificielle. L'objectif est de montrer comment le système passe d'un état de jeu encodé à une décision exploitable.

4.1 Génération et validation des coups

La génération des coups légaux constitue l'opération la plus critique du moteur. Dans le code, elle apparaît comme un générateur : l'entrée est la couleur à jouer et l'occupation courante du plateau ; la sortie est un flux de triplets `Move(from, to, arrow)` produits un par un. Pour chaque amazone, on explore les 8 directions cardinales et diagonales :

$$DIRS = \{(+1, 0), (-1, 0), (0, +1), (0, -1), (+1, +1), (-1, -1), (+1, -1), (-1, +1)\}$$

Conceptuellement, ces directions se traduisent sur le bitboard 1D par des décalages d'index. Dans l'implémentation réelle, le moteur procède toutefois plus prudemment : il reconstruit d'abord la ligne et la colonne de la case courante, applique un couple $(\Delta ligne, \Delta colonne)$, puis reconvertit en index seulement après validation des bornes. Cette étape évite de fabriquer des indices valides en apparence mais faux géométriquement.

Sur un plateau de largeur $L = n$, les directions correspondent ainsi aux variations suivantes :

- **Vertical** : Nord $(+n)$, Sud $(-n)$
- **Horizontal** : Est $(+1)$, Ouest (-1)
- **Diagonal** : NE $(+n+1)$, NO $(+n-1)$, SE $(-n+1)$, SO $(-n-1)$

4.1.1 Problème du wrapping

L'indexation linéaire crée une fausse continuité entre les bords du plateau. Par exemple, l'index 9 (colonne j , ligne 1) $+1$ donne l'index 10 (colonne a , ligne 2) : une téléportation indésirable.

Solution retenue dans le code :

- convertir l'index courant en coordonnées (`row`, `col`);
- avancer par `row += dr` et `col += dc`;
- interrompre la propagation dès que `row` ou `col` sort de $[0, n-1]$;
- ne reconstruire l'index `row * size + col` qu'après ce contrôle.

Cette solution est légèrement moins “bit trick” qu'un système de masques de bords, mais elle est plus lisible et colle exactement à la géométrie du jeu. Elle supprime le wrapping pour les mouvements horizontaux comme pour les diagonales avec une seule règle de borne.

4.1.2 Algorithme complet

Pour chaque amazone à la position P et chaque direction D :

1. Initialiser $cur = P$.
2. Convertir cur en coordonnées (row, col) puis calculer $(row + dr, col + dc)$.
3. **Bornes / anti-wrap** : si l'une des deux coordonnées sort du plateau, arrêter.
4. Reconstruire $next = row \times n + col$.
5. **Collision** : si $next$ est occupé (test bit à 1 dans `occupied`), arrêter.
6. Ajouter $next$ aux destinations valides, avancer ($cur = next$), retour à l'étape 2.
7. Pour chaque destination to , recalculer une occupation temporaire où la case de départ est libérée et la case d'arrivée devient occupée.
8. Relancer exactement le même itérateur depuis to pour générer les positions de flèche.

La procédure reste simple, mais elle est appelée partout : moteur, évaluateurs et algorithmes de recherche. Le déroulement complet du coup **e2-e4/e6** est détaillé plus loin dans la Section 6.

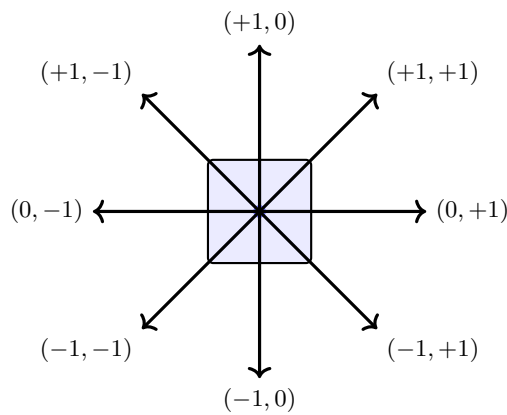


FIGURE 4.1 – Huit directions de propagation autour d'une amazone, exprimées en déplacements $(\Delta\text{ligne}, \Delta\text{colonne})$.

Cette synthèse aide à comprendre que la génération d'un coup légal combine plusieurs contrôles successifs : les bords du plateau, l'absence de collision, puis la génération des cases de tir de flèche. Elle explicite donc la différence entre un simple déplacement et un coup complet des Amazones.

4.2 Algorithmes d'intelligence artificielle

4.2.1 Minimax avec élagage $\alpha\beta$

L'algorithme Minimax [5] explore un arbre alternant des niveaux **MAX** et **MIN**. Son intérêt principal, dans notre projet, est sa bonne intégration au moteur existant : le même **Board** fournit les coups légaux, applique un coup, annule ce coup et expose l'état au scoreur. Sa limite est immédiate sur les Amazones : avec un facteur de branchement b élevé, le coût d'une recherche de profondeur d reste de l'ordre de $O(b^d)$ et croît très vite.

L'élagage $\alpha\beta$ réduit considérablement le nombre de nœuds explorés en coupant les branches dont on sait qu'elles ne peuvent pas influencer la décision finale :

Coupure si $\alpha \geq \beta$

Cette optimisation ne change pas la nature exponentielle du problème, mais elle améliore nettement le comportement pratique. Dans le projet, Minimax reste utile pour obtenir un comportement déterministe, traçable et stable sur des profondeurs modestes. La recherche est bornée par une profondeur configurable et par un budget temps, puis les feuilles sont évaluées par des heuristiques.

Techniquement, la méthode `best_move` construit d'abord la liste des coups légaux, applique chaque coup candidat, appelle `minimax(...)` puis restaure exactement le plateau. La récursion s'arrête en cas de profondeur nulle, d'absence de coups légaux ou de dépassement du budget temps. Le détail pas à pas et les tests associés sont donnés dans la Section 6.

4.2.2 Fonctions d'évaluation

Trois heuristiques sont implémentées. Elles correspondent à trois lectures différentes d'une bonne position :

Mobilité [8] :

$$score_{mob}(s) = |Moves(s, joueur)| - |Moves(s, adversaire)|$$

Territoire [7] (propagation locale depuis l'ensemble des amazones d'un camp) :

$$score_{terr}(s) = \#cases_{nous} - \#cases_{adversaire}$$

Hybride (combinaison pondérée) :

$$score_{hyb}(s) = 0.6 \cdot score_{terr}(s) + 0.4 \cdot score_{mob}(s)$$

Dans l'implémentation actuelle, l'évaluateur de territoire ne calcule pas une compétition de distances minimales case par case. Il effectue un *flood-fill* en voisinage de roi depuis toutes les amazones d'un camp, puis compte les cases atteignables. L'heuristique reste donc spatiale, mais volontairement simple et rapide.

L'évaluateur hybride combine mobilité et territoire pour mieux couvrir les différentes phases de la partie. Le code retient un compromis fixe 0.6/0.4 : la pondération est stable et simple à utiliser, mais elle n'est pas encore ajustée dynamiquement selon la position.

4.2.3 Monte Carlo Tree Search (MCTS)

MCTS [2] explore l'arbre de jeu par simulations successives. Cette approche est intéressante pour les Amazones car elle supporte mieux les très grands facteurs de branchement qu'une recherche Minimax profonde. Au lieu de fixer d'emblée une profondeur, MCTS investit progressivement le budget temps dans les branches les plus prometteuses :

1. **Sélection** : descendre dans l'arbre en choisissant les nœuds selon une variante du critère UCT [6]. Dans notre code, chaque nœud stocke ses victoires du point de vue du joueur qui doit jouer à ce nœud. La sélection inverse donc le taux de victoire de l'enfant pour raisonner du point de vue du parent, puis ajoute un terme d'exploration.
2. **Expansion** : ajouter un nœud fils non exploré.
3. **Simulation** (rollout) : jouer aléatoirement jusqu'à la fin de partie.
4. **Rétropropagation** : mettre à jour les statistiques (w_i, n_i) de chaque nœud traversé.

L'avantage principal de cette approche est sa nature *anytime* : plus on lui accorde de temps, plus les statistiques se raffinent, sans profondeur fixe imposée à l'avance. En contrepartie, la qualité du résultat dépend fortement de la politique de simulation et du réglage exploration/exploitation. Dans notre implémentation, les rollouts restent simples ; cela préserve la lisibilité du code, mais limite la qualité stratégique des estimations.

4.2.4 Comparaison critique des approches

Minimax et MCTS ne répondent pas au même besoin. Minimax convient mieux quand on veut une décision stable et justifiable par une heuristique explicite. MCTS supporte mieux les grands facteurs de branchement et utilise le temps disponible de façon plus souple, au prix d'une décision moins lisible. Garder les deux moteurs permet donc de comparer deux stratégies de décision sous contrainte de temps.

5 Architecture du système

L'architecture du système repose sur une idée simple : un même noyau métier doit servir tous les modes d'utilisation. La CLI, la GUI, l'IA, la sauvegarde et le réseau ne redéfinissent pas les règles ; ils réutilisent les mêmes opérations de validation et les mêmes structures de données.

La Figure 5.1 ne représente pas une hiérarchie de classes. Elle résume le chemin fonctionnel principal d'une action utilisateur dans le système.

$$Vue (CLI / GUI) \rightarrow GameController \rightarrow Board \rightarrow Player \rightarrow Move$$

FIGURE 5.1 – Workflow global du système : de l'action utilisateur jusqu'au coup validé.

Ce schéma résume le flux principal du projet. La vue transmet une intention d'action au contrôleur, le contrôleur s'appuie sur **Board** pour valider l'état et délègue ensuite l'obtention du coup au joueur courant, qu'il soit humain, IA ou réseau. Le résultat reste un objet **Move** unique, ce qui garantit un traitement homogène entre tous les modes de jeu.

Le rôle de la vue doit être précisé. **ViewCli** lit des commandes textuelles, affiche l'état de la partie et relaie les demandes utilisateur. **ViewGui** capte des événements GTK, dessine le plateau et orchestre les interactions visuelles. Aucune de ces vues ne décide elle-même si un coup est légal. Elles collectent des intentions d'action, puis les transmettent à **GameController**. Cette séparation est essentielle : elle évite d'avoir une règle implémentée une fois en CLI et une autre en GUI.

Dans l'implémentation actuelle, cette chaîne reste pilotée par **GameController**, mais une partie du traitement passe désormais par les modules auxiliaires de **controller/engine**. Le pipeline reste donc volontairement simple dans le rapport, tout en correspondant à une implémentation devenue plus modulaire.

La Figure 5.2 joue le même rôle pour la décision artificielle : elle présente un enchaînement fonctionnel, pas une liste de classes.

Board → *legal moves* → *search engine* → *best move*

FIGURE 5.2 – Pipeline de décision de l'intelligence artificielle.

Le pipeline IA clarifie le rôle des différents composants. Le plateau fournit l'état courant et les coups légaux. Un moteur de recherche exploite ensuite cet espace de décision. Selon le moteur choisi, la décision repose soit sur une évaluation heuristique, soit sur des simulations. Le résultat final reste un coup concret, directement exploitable par le contrôleur.

5.1 Architecture générale

5.1.1 Pattern MVC

L'architecture suit le patron **Modèle-Vue-Contrôleur** (MVC) [3], avec l'objectif de séparer la logique métier, les mécanismes d'orchestration et les interfaces utilisateur :

Couche	Responsabilité	Répertoires principaux
Model	Données et logique métier	model/
View	Affichage et interaction utilisateur	view/ui/
Controller	Orchestration et flux de jeu	controller/, controller/engine/, controller/network/

TABLE 5.1 – Découpage MVC du projet

Dans notre implémentation, le modèle encapsule les invariants du jeu et les structures de données partagées ; la vue se limite, dans l'intention, à la présentation et à la collecte des interactions ; le contrôleur convertit enfin ces interactions en opérations métier. Ce découpage a été retenu pour une raison concrète : les mêmes règles doivent être appelées depuis la CLI, la GUI, l'IA et le réseau. Sans cette séparation, la validation des coups et la gestion de l'état auraient été dupliquées dans plusieurs interfaces.

Dans les faits, le contrôleur centralise toutefois une part importante des dépendances entre couches. Il instancie le plateau, les joueurs humains, IA ou réseau, délègue l'affichage à la CLI ou à la GUI, et coordonne la sauvegarde, le chargement, l'historique, le chronométrage et les échanges réseau. Ce choix a simplifié l'intégration initiale du projet, mais il a un coût clair : **GameController** reste le point où se concentrent plusieurs causes de changement. Le code actuel a commencé à extraire certaines responsabilités vers des modules spécialisés de **controller/engine**, ce qui améliore la lisibilité sans supprimer encore cette centralisation.

L'architecture générale reste simple. Le noyau métier repose sur **Board**, **Move**, la hiérarchie des joueurs et les moteurs d'IA. La persistance, le réseau et l'internationalisation jouent un rôle transverse. La CLI et la GUI sont deux entrées vers ce même noyau. Dans le code actuel, la couche contrôleur ne se limite plus à **game_controller.py** : une partie de la logique a été déplacée dans **controller/engine**. **GameController** reste toutefois la façade centrale. Cette factorisation améliore la lecture locale, mais le couplage reste fort.

5.1.2 Découpage en modules

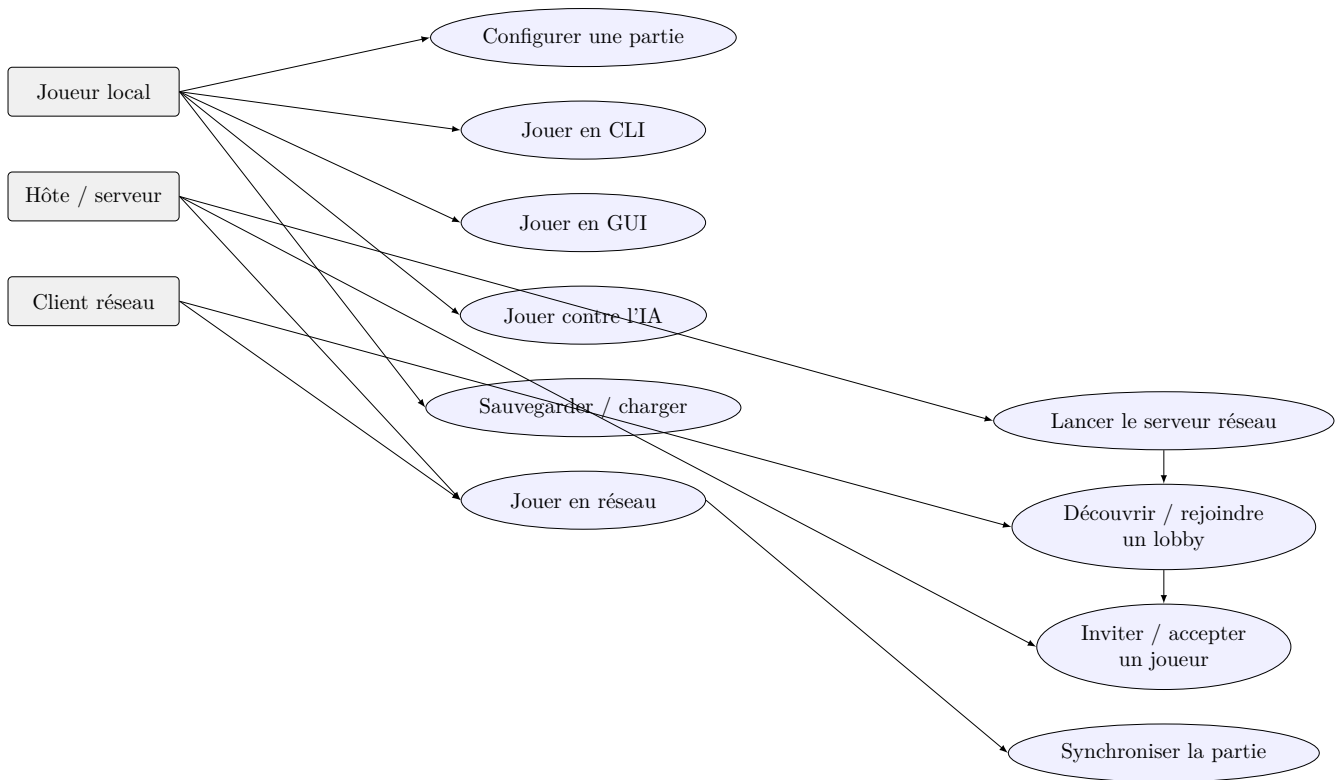


FIGURE 5.3 – Diagramme de cas d'utilisation des fonctionnalités principales, incluant explicitement le serveur réseau et le lobby.

Le diagramme de cas d'utilisation montre que l'IA, le réseau, la sauvegarde et la GUI prolongent le scénario central de jeu au lieu de constituer des applications séparées. Il fait aussi apparaître explicitement la partie *serveur réseau*, qui ne se limite pas au transport des coups : démarrage du serveur, gestion du lobby, invitations et synchronisation des sessions font partie intégrante du système.

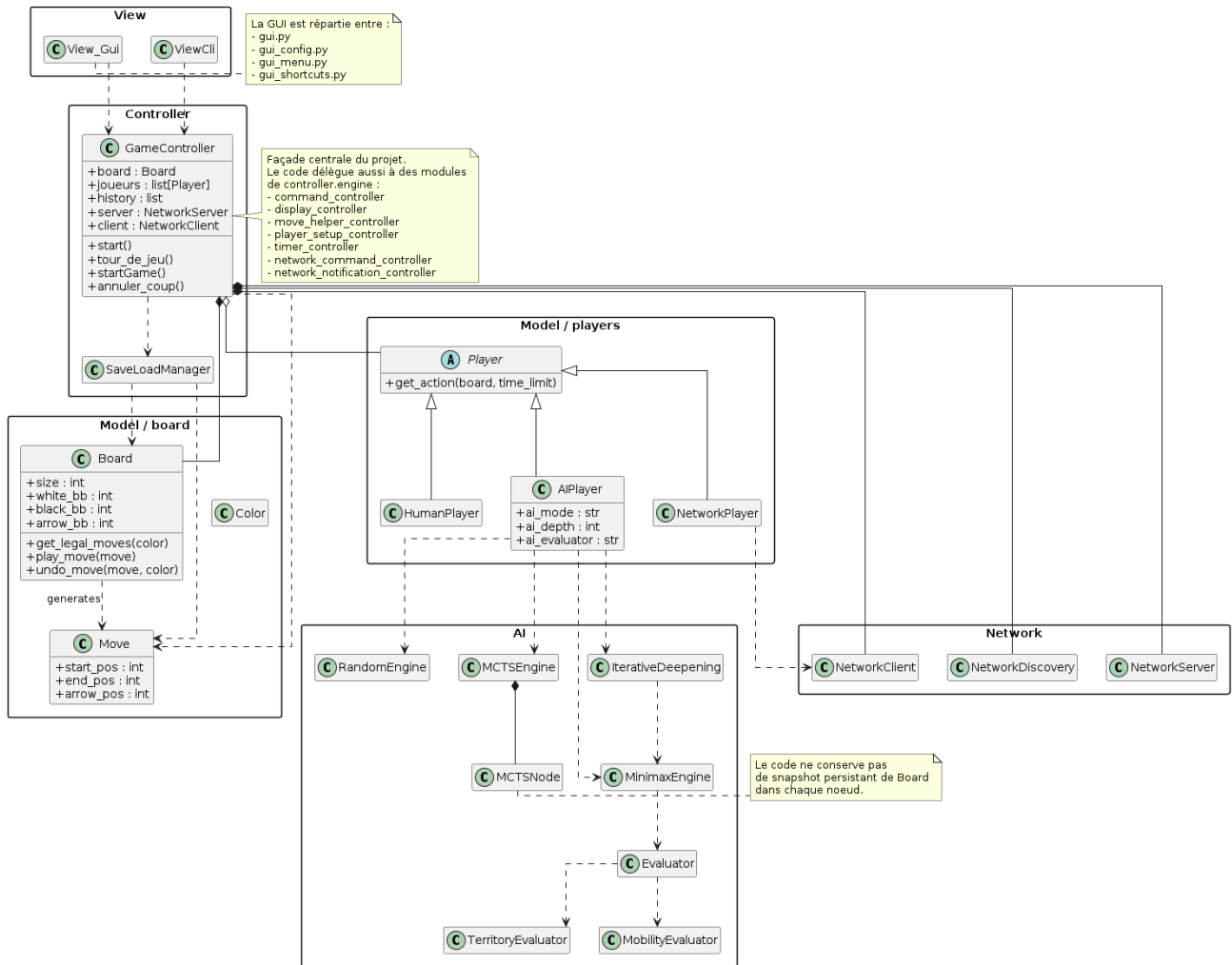


FIGURE 5.4 – Diagramme de classes UML du noyau structurant du projet.

Le diagramme de classes a été volontairement limité aux classes réellement structurantes du projet. On y retrouve le noyau métier (**Board**, **Move**, **Player**), les spécialisations de joueurs, les moteurs d'IA, les vues CLI et GUI, ainsi que les services de persistance et de réseau manipulés par le contrôleur. Ce choix permet de garder une lecture claire du système sans mélanger structure objet et découpage modulaire.

La séparation récente de **controller/engine** n'apparaît donc pas sous la forme de nouvelles classes dans ce diagramme. C'est un point important : `command_controller.py`, `display_controller.py`, `timer_controller.py` et les autres fichiers du dossier factorisent la logique du contrôleur, mais ils ne définissent pas pour autant des objets métier autonomes. Le diagramme de classes doit rester fidèle à la structure objet réelle ; la factorisation modulaire du contrôleur est, elle, expliquée dans le texte d'architecture.

5.1.3 Module Model

- `board/board.py` : classe **Board** encapsulant les trois bitboards, la validation des coups et les transitions d'état principales du plateau.
- `board/move.py` : classe **Move** (triplet `start_pos`, `end_pos`, `arrow_pos`) avec conversion algébrique $\leftrightarrow$ index.
- `players/` : hiérarchie de joueurs — **HumanPlayer**, **AIPlayer**, **NetworkPlayer**.
- `ai/` : moteurs d'IA (`minimax_engine.py`, `iterative_deepening.py`, `mcts_engine.py`, `random_engine.py`) et évaluateurs (`mobility_evaluator.py`, `territory_evaluator.py`, combinaison hybride dans `evaluator.py`).

5.1.4 Module Controller

- `engine/game_controller.py` : classe centrale **GameController**, responsable de la boucle de jeu et de l'orchestration applicative.
- Modules de commande et d'orchestration locale : `engine/command_controller.py`, `display_controller.py`, `move_helper_controller.py`, `player_setup_controller.py` et `timer_controller.py`.
- `engine/network_command_controller.py` et `network_notification_controller.py` : modules dédiés à une

partie des échanges et notifications réseau, toujours branchés sur le même objet `GameController`.

- `save_load_manager.py` : service de sérialisation et de reconstruction des parties au format ASCII, avec contrôle de cohérence et checksum anti-triche.
- `network/` : services réseau TCP/UDP pour la découverte, la connexion, les invitations, la synchronisation des coups et la gestion des sessions distantes.

La factorisation engagée dans `controller/engine` améliore la lisibilité et limite la croissance du fichier principal. Elle ne constitue pas encore un vrai découplage : les modules auxiliaires restent branchés sur le même état contrôleur, et plusieurs responsabilités restent concentrées autour de `GameController`. Le projet est donc passé d'un contrôleur monolithique à une façade centrale entourée de modules spécialisés. Le gain est réel, mais incomplet.

5.1.5 Module View

- `ui/cli.py` : interface en ligne de commande avec shell interactif, complétion, affichage du plateau, aide contextuelle et commandes réseau.
- `ui/gui.py`, `ui/gui_config.py`, `ui/gui_menu.py` et `ui/gui_shortcuts.py` : interface graphique GTK4, écran de configuration, affichage du plateau, menus, raccourcis clavier personnalisables, sauvegarde/chargement, interactions réseau et gestion du lobby graphique.

La coexistence de deux vues complètes n'est pas un simple confort d'utilisation. C'est un test architectural : si le moteur et le contrôleur sont correctement conçus, la même logique métier doit pouvoir être pilotée par une boucle CLI, par des callbacks GTK et par des événements réseau. Cette pluralité d'interfaces distingue le projet d'un prototype limité à un mode d'interaction unique.

5.1.6 Interfaces principales entre modules

Au-delà du découpage en dossiers, quelques interfaces structurent réellement le projet :

- **Interface du plateau** : `Board` expose les opérations essentielles au reste du système, comme `get_legal_moves`, `is_valid_move`, `make_move`, `undo_move` et `get_board_array`. Cette API est utilisée à la fois par le contrôleur, les IA et la vue.
- **Interface de persistance** : `SaveLoadManager` joue le rôle de service transversal. Le contrôleur et la GUI s'appuient dessus pour sérialiser et reconstruire une partie sans dupliquer la logique.
- **Interface joueur** : la hiérarchie des joueurs permet au contrôleur de traiter de manière homogène un humain, une IA ou un joueur réseau, même si la récupération effective du coup diffère selon le type.
- **Interface vue** : la CLI et la GUI ne calculent pas les coups elles-mêmes ; elles transmettent des intentions d'action à la façade `GameController`, qui délègue ensuite une partie du traitement aux modules spécialisés de `controller/engine`.

Cette séparation reste partielle : `GameController` conserve encore beaucoup de responsabilités. Nous avons accepté ce compromis pour intégrer rapidement les sous-modules pendant le développement. La dette principale est claire : une future version devrait extraire la gestion des tours, le réseau, la persistance et l'orchestration GUI/CLI dans des services distincts.

5.1.7 Diagrammes de séquence

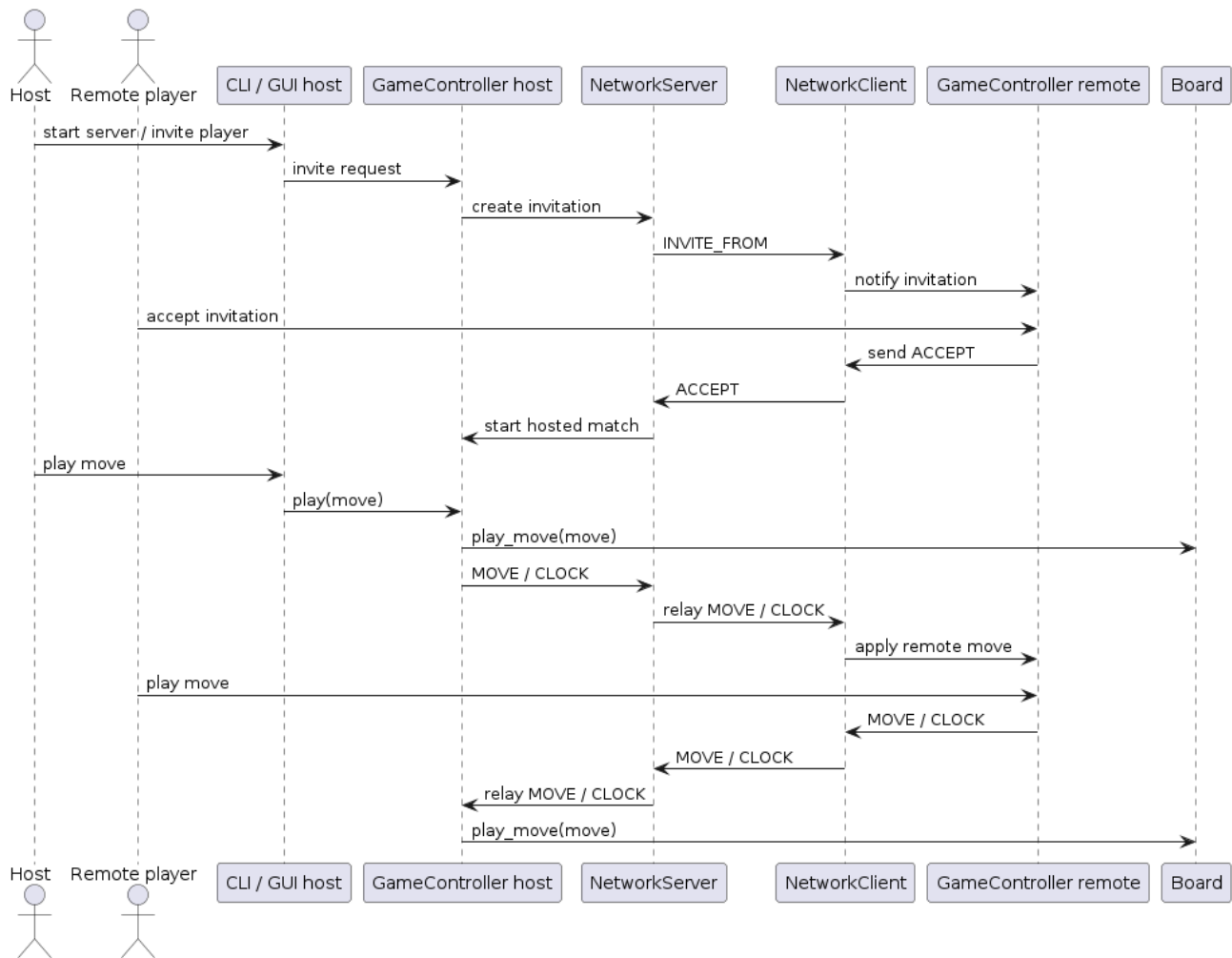


FIGURE 5.5 – Séquence : CLI humain vs humain

Ce diagramme retrace un tour local en CLI entre deux joueurs humains. La vue transmet la commande au **GameController**. Le contrôleur vérifie le coup, met à jour le plateau et renvoie l'état affiché. Le second joueur suit le même chemin. La CLI sert donc d'interface d'entrée, tandis que la logique reste portée par le contrôleur et le modèle.

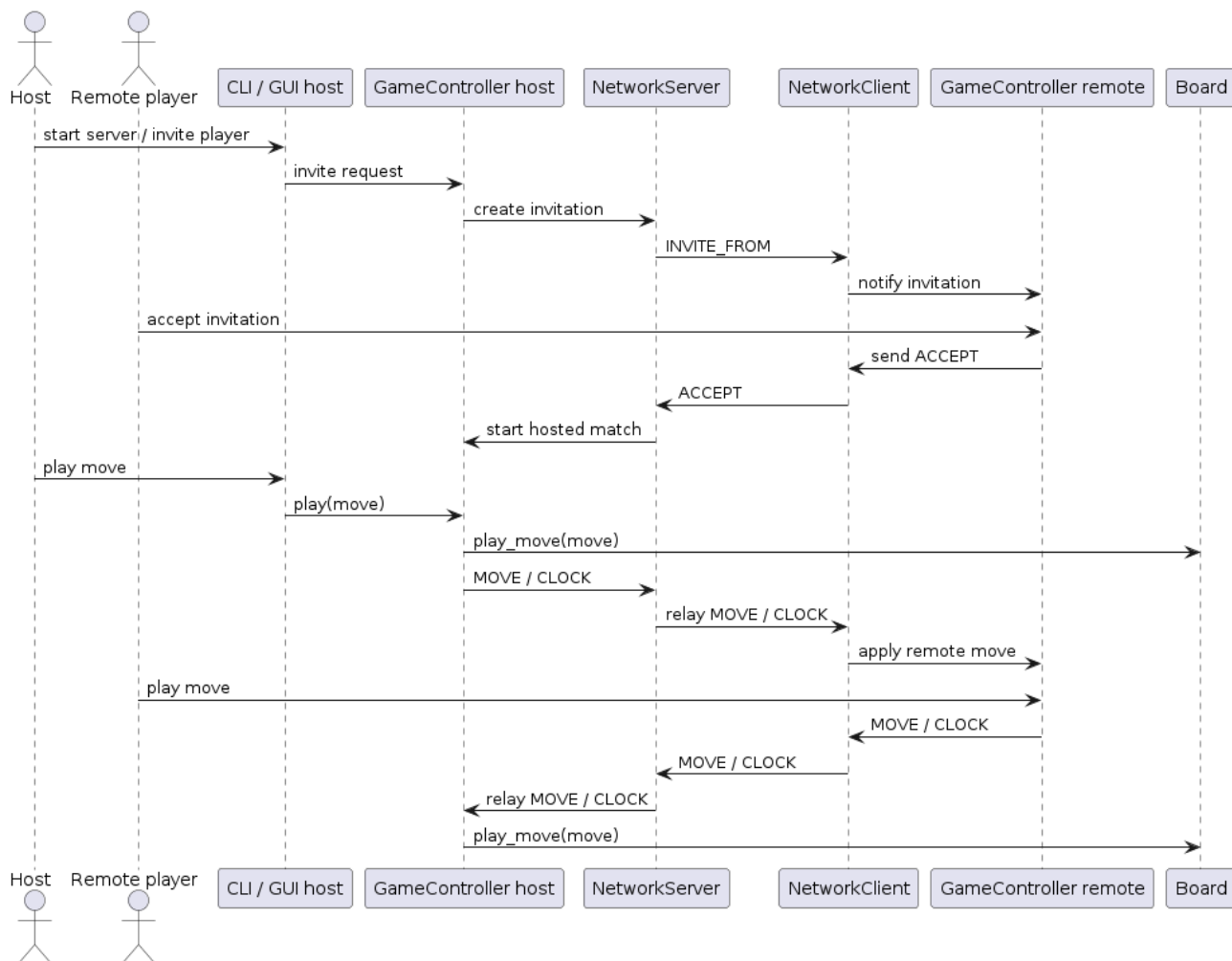


FIGURE 5.6 – Séquence : scénario réseau

La Figure 5.6 montre le même enchaînement lorsqu'un coup vient du réseau. Avant d'atteindre le plateau, le message passe par les composants de découverte, de connexion et de synchronisation. Une fois reçu, il est traité par le **GameController** comme un coup ordinaire. Le réseau ajoute donc une couche de transport, sans modifier les règles du jeu.

5.1.8 Architecture réseau

Le mode réseau repose sur une architecture client–serveur utilisant **socket** [11], avec séparation explicite entre découverte, connexion persistante et gestion de sessions. Cette séparation évite de confondre l'identification du serveur, l'entrée dans le lobby et la synchronisation stricte d'une partie.

- **Découverte** : broadcast UDP pour localiser le serveur sur le réseau local.
- **Connexion** : canal TCP persistant pour l'échange ordonné des commandes de partie.
- **Lobby** : enregistrement des joueurs connectés, gestion d'identifiants, invitations et état d'attente.
- **Session** : synchronisation d'une partie active, transmission des coups, de l'undo, des événements de fin de partie et mise à jour du scoreboard.

Le choix UDP/TCP répond directement au besoin. UDP convient à la découverte : le service doit être visible à faible coût, et la perte d'un paquet reste tolérable. TCP est nécessaire pour la partie elle-même : un coup perdu, dupliqué ou désordonné casserait la cohérence des deux plateaux. Le protocole réseau doit donc être lu comme un mécanisme de synchronisation d'état.

Cette partie correspond directement aux besoins **F38**, **F39** et **F40**. Le contrôleur utilise **NetworkDiscovery** pour lister les serveurs. Il s'appuie ensuite sur **NetworkServer** pour accepter plusieurs connexions et sur **NetworkClient** pour rejoindre une partie existante.

Le protocole repose sur des messages textuels simples, mais il dépasse largement le noyau minimal PING, PONG, START, MOVE et QUIT. Il couvre aussi plusieurs familles d'événements :

- notifications de lobby : **PLAYER_CONNECTED** ;

- invitations : `INVITE_*` ;
- lancement de partie : `HOST_START_GAME` ;
- synchronisation d'horloge : `CLOCK` ;
- undo réseau : `UNDO_REQ`, `UNDO_OK`, `UNDO_NO` ;
- fin de partie : `GAMEOVER` ;
- synchronisation de sauvegarde en mode hôte-client : `LOAD_STATE`.

Ce choix facilite nettement les tests et le débogage, au prix d'un formalisme limité et d'un typage faible des échanges.

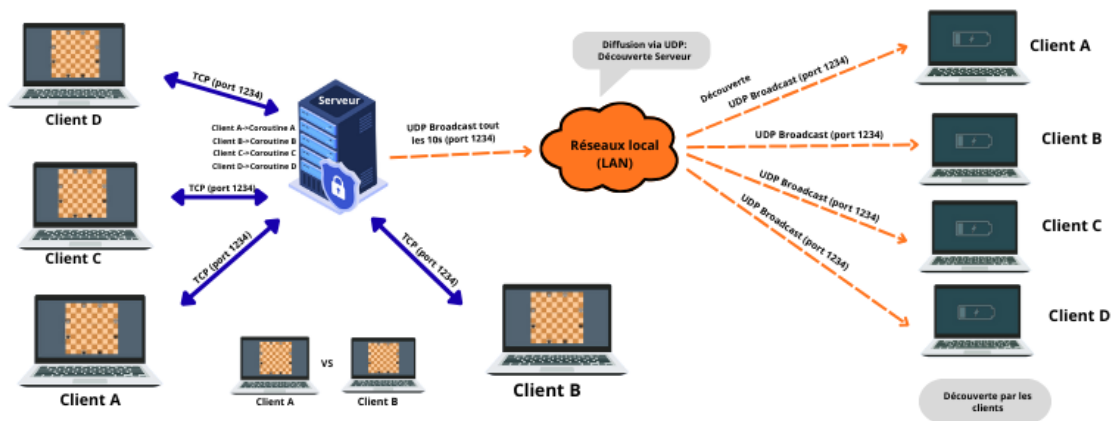


FIGURE 5.7 – Architecture réseau client-serveur

Ce schéma représente le flux de communication entre découverte, connexion et partie effective. Il illustre un choix architectural important : la couche réseau s'ajoute au moteur existant sans le remplacer. La validation des coups reste locale au moteur, tandis que le réseau transporte des intentions et des états synchronisés. Cette approche réduit le risque de divergence de règles entre mode local et mode distant, mais elle impose une vigilance particulière sur la synchronisation des états.

L'architecture actuelle est plus riche qu'un schéma client unique contre serveur. Le serveur maintient un lobby multi-clients, des invitations, un joueur hôte, des sessions actives entre deux clients ou entre l'hôte et un client, ainsi qu'un scoreboard persistant pendant la durée de vie du serveur. Cette dimension est importante à expliciter dans le rapport : elle montre que le système réseau ne se limite pas à transférer des coups, mais gère aussi un cycle de vie de partie, des rôles, des transitions d'état et des déconnexions. En outre, le code distingue explicitement deux cas d'usage : les matchs *hosted* (hôte contre client) et les matchs *relayed* (client contre client via le serveur), qui n'autorisent pas exactement les mêmes opérations, notamment pour la synchronisation de sauvegarde.

Cette synchronisation constitue d'ailleurs l'une des limites structurelles du projet. Le réseau est traité selon un modèle principalement synchrone côté contrôleur, alors que la GUI introduit naturellement une logique événementielle et asynchrone. Cette tension entre boucles bloquantes de type CLI et callbacks graphiques rend la conception plus fragile qu'un système entièrement événementiel. Dans une version industrielle, on privilégierait probablement un protocole de messages typés et une couche d'orchestration asynchrone explicitement séparée de la logique métier.

5.1.9 Intégration des fonctionnalités avancées

La liste complète des quarante besoins figure en annexe. Dans le corps du rapport, nous montrons surtout comment les fonctionnalités avancées reposent sur le même noyau technique.

- Les bitboards et la génération de coups forment le socle commun du moteur.
- Les moteurs d'IA réutilisent directement `Board`, `Move` et les opérations de jeu.
- Le réseau transporte des commandes et des états synchronisés, mais ne redéfinit pas les règles.
- La GUI pilote le même contrôleur et les mêmes objets métier que la CLI.
- La persistance reconstruit ce même état, ce qui évite de multiplier les formats et les chemins de traitement.

Cette organisation explique pourquoi le projet reste cohérent malgré la diversité des modes d'utilisation. Les fonctionnalités avancées ne contournent pas le moteur central ; elles l'exploitent et l'étendent.

5.1.10 Internationalisation

Le projet supporte l'anglais (par défaut) et le français via `gettext` [10]. La langue peut être imposée au lancement ou déduite de la locale système. Les catalogues sont rangés dans `locales/en/` et `locales/fr/`. Ce choix garde la traduction dans un service transversal au lieu de la disperser dans les modules métier.

5.2 Préoccupation de qualité logicielle

Dès la phase de conception, une attention particulière a été portée à la testabilité et à la cohérence des composants. Le moteur de jeu, la persistance, les services réseau et les vues ont été séparés autant que possible afin de limiter les dépendances circulaires et de faciliter la validation ultérieure. Cette préoccupation explique notamment le rôle des services transversaux comme **SaveLoadManager** et l'importance donnée à des interfaces stables entre le contrôleur, le plateau et les différents types de joueurs.

Cette logique de qualité ne supprime pas les défauts du système. Elle aide surtout à lire les choix faits dans le projet : compromis entre extensibilité, délai de réalisation, lisibilité et performance.

Décision	Gain principal	Coût ou dette associée
Bitboards	Représentation compacte, tests d'occupation rapides, bonne adéquation au moteur et à l'IA	Lisibilité plus faible qu'une matrice 2D, indexation plus abstraite
Coexistence Mini-max / MCTS	Comparaison de stratégies de décision complémentaires sous une API commune	Maintenance plus exigeante, réglages d'heuristiques multiples
Contrôleur central	Intégration rapide des fonctionnalités transversales	Concentration des responsabilités et maintenabilité réduite
Protocole réseau textuel	Débogage simple, inspection aisée des échanges	Typage faible, validation plus fragile qu'avec des messages structurés

TABLE 5.2 – Synthèse des décisions d'ingénierie structurantes du projet.

Synthèse des décisions d'ingénierie.

6 Explication approfondie

Cette section isole deux fonctionnalités effectivement réalisées et centrales dans le projet. Le choix retenu n'est pas arbitraire : la génération des coups bitboard constitue le noyau fonctionnel du moteur, tandis que Minimax avec élagage $\alpha\beta$ est le moteur d'IA le plus directement explicable et le mieux intégré au code actuel.

6.1 Fonctionnalité 1 — Génération des coups avec bitboards

Objectif. Produire tous les coups légaux d'un joueur sous la forme **Move(start, end, arrow)**. Cette fonctionnalité est partagée par le moteur, la validation locale, les IA et le contrôleur ; une erreur ici affecte donc immédiatement tout le système.

Conception. Le code s'appuie sur trois bitboards spécialisés et sur une occupation agrégée **occupied**. Les amazones du joueur sont extraites une par une via le bit de poids faible, puis le même itérateur de déplacement de reine est réutilisé deux fois : d'abord pour les destinations de déplacement, puis pour les cibles de flèche après reconstruction d'une occupation temporaire.

Algorithme détaillé. Entrée : la couleur à jouer et l'état courant du plateau. Sortie : un flux de coups complets.

Pour le coup **e2-e4/e6**, l'exécution suit exactement les étapes suivantes :

1. sélectionner l'amazone de départ **e2** ;
2. propager dans la direction choisie en contrôlant à chaque pas les bornes ligne/colonne et l'absence de collision ;
3. retenir **e4** seulement si le chemin **e2** $\rightarrow$ **e4** est libre ;
4. reconstruire **occupied_after_move** en retirant **e2** puis en occupant **e4** ;
5. relancer exactement le même itérateur depuis **e4** pour les tirs de flèche ;
6. retenir **e6** seulement si cette seconde trajectoire est libre ;
7. produire finalement **Move(e2, e4, e6)**.

Le *wrapping* est évité par le contrôle explicite de **row** et **col** avant reconstruction de l'index. L'occupation intervient donc deux fois : d'abord pour filtrer les destinations, puis sous forme modifiée pour filtrer les cases de flèche.

Déroulement technique. L'implémentation repose sur deux boucles imbriquées de propagation. La première parcourt toutes les destinations atteignables depuis une amazone donnée dans l'occupation courante. La seconde repart de chaque destination candidate avec une occupation recalculée, où la case de départ est redevenue libre et la case d'arrivée est désormais occupée. Cette mise à jour intermédiaire est le point décisif de la fonction : elle évite d'accepter un tir de flèche à travers l'amazone déplacée, tout en autorisant un tir sur l'ancienne case de départ lorsque les règles

le permettent. La sortie finale n'est donc pas une simple liste de destinations, mais un ensemble de triplets complets (départ, arrivée, flèche) déjà compatibles avec l'état réel du plateau après déplacement.

Intégration dans le système. Cette fonctionnalité est fournie par `Board.get_legal_moves(color)`. Elle est utilisée directement par `Board.is_valid_move`, par `MinimaxEngine` et `MCTSEngine`, et indirectement par le contrôleur lorsqu'il doit décider si un joueur peut encore jouer.

Tests associés. Les tests vérifient notamment qu'un coup bloqué n'est pas produit, qu'un déplacement en bord de plateau n'induit pas de *wrapping*, qu'un coup généré puis annulé restaure exactement les trois bitboards, et que l'ancienne case de départ peut devenir une cible de flèche après libération.

6.2 Fonctionnalité 2 — IA Minimax avec élagage $\alpha\beta$

Objectif. Choisir un coup légal de bonne qualité sous contrainte de profondeur et de temps. Dans le projet, ce moteur sert de base déterministe pour l'IA et de référence pour comparer le comportement de MCTS.

Conception. `AIPlayer` ne réalise pas lui-même la recherche. Il instancie `MinimaxEngine` selon le mode demandé, puis lui transmet le plateau et la couleur du joueur. Le moteur réutilise directement les primitives du plateau : génération des coups, application, annulation et évaluation heuristique.

Algorithme détaillé. La méthode `best_move(board, color)` matérialise d'abord tous les coups légaux du joueur courant. Pour chaque coup candidat, elle :

1. applique le coup via `board.play_move(move)` ;
2. appelle récursivement `minimax(...)` avec alternance MAX/MIN ;
3. coupe une branche si $\alpha \geq \beta$;
4. restaure l'état exact via `board.undo_move(move, color)` ;
5. conserve le coup associé au meilleur score.

La récursion s'arrête dans trois cas : profondeur nulle, absence de coups légaux pour le joueur courant, ou dépassement du budget temps. Dans les feuilles, le score provient de `Evaluator.evaluate(board, color, mode)`.

Déroulement technique. Le moteur commence par figer l'ensemble des coups légaux du joueur courant, puis les évalue un par un dans une boucle principale. Chaque itération suit le même cycle : appliquer le coup sur le plateau partagé, lancer l'exploration récursive avec alternance MAX/MIN, mettre à jour les bornes α et β , puis restaurer immédiatement l'état exact du plateau. Le meilleur coup n'est conservé qu'après cette restauration, ce qui garantit que l'itération suivante repart d'un état propre. Dans le code, cette discipline « appliquer, explorer, annuler » est essentielle : elle évite la copie systématique du plateau, garde la mémoire sous contrôle et permet d'utiliser le même moteur sur tous les états produits par le contrôleur.

Intégration dans le système. Le couplage avec le reste du projet reste simple. `AIPlayer.get_action(...)` construit le moteur. `MinimaxEngine.best_move(...)` interroge ensuite `Board`. Le coup retenu est renvoyé au contrôleur comme un coup humain. Le contrôleur n'a donc pas besoin de connaître la stratégie interne du moteur.

Limites concrètes et comportement observé. Ce moteur est fiable et reproductible, mais il souffre rapidement de l'explosion combinatoire. Les mesures du rapport montrent déjà qu'à profondeur 2, les grands plateaux sortent du cadre interactif. En pratique, Minimax reste donc pertinent pour des profondeurs modestes, pour l'approfondissement itératif et pour des scénarios où l'on veut une décision explicable par une heuristique.

Tests associés. Les tests associés vérifient qu'un coup renvoyé par Minimax appartient bien à l'ensemble des coups légaux, qu'un plateau sans coup légal renvoie `None`, que le moteur reste borné par le budget temps, et que l'appel à `best_move` ne corrompt pas durablement l'état du plateau grâce au couple `play_move/undo_move`.

7 Implémentation

7.1 Moteur de jeu et structures internes

7.1.1 Représentation mémoire

En Python, les entiers sont de précision arbitraire, ce qui permet de représenter un plateau allant jusqu'à $11 \times 11 = 121$ cases dans une seule valeur entière. Le moteur maintient trois bitboards spécialisés : un pour les amazones blanches, un pour les amazones noires et un pour les flèches. Cette séparation est un invariant central du modèle : un bitboard ne mélange jamais plusieurs types d'objets.

L'occupation totale est calculée par une simple opération OR :

```
occupied = white_bb | black_bb | arrow_bb
```

Ce choix a deux effets directs sur le moteur. D’abord, tester une case coûte une seule opération binaire : on forme le masque `1 << pos`, puis on vérifie `occupied & mask`. Ensuite, déplacer une amazone ou poser une flèche revient à modifier quelques bits dans les trois bitboards, sans recopier un tableau complet. C’est exactement le type d’opération répété par `get_legal_moves`, `play_move`, `undo_move` et les moteurs d’IA.

L’exemple de `d4` sur un plateau 10×10 rend cela concret. Le code numérote les cases en *row-major* depuis le coin supérieur gauche ; `d4` correspond donc à la colonne 3 et à la ligne interne 6, soit l’index 63. Le masque `1 << 63` pointe exactement cette case. On peut alors tester son occupation via `occupied & (1 << 63)`, l’activer via `bitboard | (1 << 63)` et la désactiver par XOR si le bit est déjà actif. Le gain n’est donc pas limité à la mémoire : une case devient une cible binaire que le moteur manipule directement.

7.1.2 Génération des coups légaux — Implémentation détaillée

`get_legal_moves(color)` reprend la logique détaillée dans la Section 6. Ici, le point important est son rôle central : validation, IA et détection de fin de partie reposent sur cette même méthode.

7.1.3 Application et annulation de coups

L’application d’un coup modifie directement les bitboards via XOR et OR : retrait de l’amazone sur la case de départ, activation de la case d’arrivée, puis ajout de la flèche. L’annulation suit l’ordre inverse. Cet ordre garantit notamment qu’une flèche peut retomber légalement sur l’ancienne case de l’amazone si elle est redevenue libre au bon moment.

7.1.4 Validation des coordonnées

La conversion algébrique $\rightarrow$ index inclut une validation stricte des bornes pour prévenir les erreurs silencieuses. Le moteur sépare le calcul de la colonne, le calcul de la ligne et le contrôle de compatibilité avec la taille du plateau. En cas d’entrée invalide, il lève explicitement une exception plutôt que de corriger ou d’ignorer la valeur, ce qui protège la CLI, la GUI et le chargement de parties contre les coordonnées incohérentes.

7.2 Implémentation de l’intelligence artificielle

L’intelligence artificielle du jeu repose sur plusieurs stratégies complémentaires, sélectionnées explicitement dans le code en fonction du mode choisi pour le joueur.

7.2.1 Architecture de l’IA

Le joueur IA (`AIPlayer`) ne contient pas lui-même les algorithmes de recherche. Il choisit un moteur selon le mode configuré (`random`, `minimax`, `iterative`, `mcts`) et lui délègue la production du coup. Le terme « hybride » désigne ici l’évaluateur, pas un moteur combinant directement Minimax et MCTS. L’option ML visible dans le lancement n’est pas encore reliée à un moteur d’apprentissage réel.

7.2.2 Sélection du moteur

Ce `dispatch` simplifie l’ajout futur de nouveaux moteurs et évite d’exposer au contrôleur des détails propres à Minimax ou à MCTS.

7.2.3 Implémentation Minimax avec élagage $\alpha\beta$

Dans le code, `MinimaxEngine.best_move` itère sur les coups produits par `board.get_legal_moves(color)`, applique chaque coup, appelle `minimax(...)` puis restaure l’état avec `board.undo_move(move, color)`. Le détail algorithmique et les scénarios de test sont donnés dans la Section 6. Le point d’implémentation utile ici est l’absence de copie complète du plateau à chaque nœud : le moteur réutilise le même objet `Board` par application et annulation successives.

7.2.4 Implémentation MCTS

Le MCTS [2] est particulièrement adapté aux jeux à grand facteur de branchement comme les Amazones. Chaque nœud mémorise le joueur à traiter, le coup ayant conduit à ce nœud, le nombre de visites, le nombre de simulations gagnantes, les enfants déjà explorés et les coups encore inexplorés.

La sélection d’un enfant s’appuie sur une variante de UCT. Le code stocke les victoires du point de vue du joueur du nœud enfant. Pour choisir depuis le parent, il convertit donc ce score en taux de succès du parent, puis ajoute le terme d’exploration classique. Ce détail est important : il explique pourquoi la formule du code n’est pas écrite exactement comme la version canonique des articles.

La boucle MCTS s’exécute pendant le budget temps alloué :

1. **Sélection** : descendre en choisissant le fils avec le meilleur UCT.
2. **Expansion** : ajouter un fils non exploré.
3. **Simulation** : partie aléatoire jusqu’à la fin (rollout).
4. **Rétropropagation** : mettre à jour wins et visits sur tout le chemin.

Le point essentiel d'implémentation est le suivant : les nœuds MCTS ne conservent pas de copie complète du plateau. Le moteur reconstruit un état de travail à partir du plateau racine et rejoue les coups au fil de l'itération. Ce choix réduit la pression mémoire, au prix d'un surcoût de recomputation.

7.2.5 Approfondissement itératif et budget temps

En plus de Minimax et MCTS, le projet inclut une stratégie d'**approfondissement itératif**. Le principe consiste à lancer successivement des recherches Minimax de profondeur croissante tant que le budget temps n'est pas épuisé. Cette approche présente deux avantages concrets :

- elle garantit qu'un **meilleur coup connu** est toujours disponible, même si le temps est écoulé brutalement ;
- elle permet d'utiliser efficacement le temps disponible sans fixer à l'avance une profondeur trop ambitieuse ou trop conservatrice.

Cette stratégie améliore la robustesse pratique du moteur sur les grands plateaux tout en restant compatible avec les budgets temps imposés au joueur artificiel.

7.2.6 Évaluateur territorial (BFS)

L'évaluateur de territoire, inspiré de Lieberum [7], repose dans notre version sur une propagation locale depuis toutes les amazones d'un joueur, en voisinage de roi, à travers les cases libres. Il ne cherche donc pas encore à calculer une véritable compétition de distances minimales entre les deux camps ; il mesure plutôt l'étendue des zones encore atteignables sans traverser les obstacles. Cette approximation reste intéressante car elle capte déjà la structure spatiale du plateau pour un coût raisonnable, mais elle doit être comprise comme une heuristique d'ingénierie et non comme une modélisation exacte du contrôle territorial.

7.2.7 Gestion du temps

L'IA dispose d'un budget temps configurable (`--ai-time`, défaut 5s). Le Minimax avec approfondissement itératif augmente progressivement la profondeur tant que le temps le permet, ce qui garantit qu'un coup valide reste toujours disponible. Le MCTS, lui, s'arrête naturellement quand le budget est épuisé.

7.3 Persistance, interfaces et intégration

7.3.1 Sauvegarde et chargement des parties

Le service de persistance repose sur un format ASCII structuré en sections. Un fichier de sauvegarde contient les paramètres de partie, l'état du plateau, l'historique des coups et, lorsque cela s'applique, un checksum. Ce choix garde le format lisible, facile à déboguer et indépendant d'un format binaire opaque.

Le chargeur accepte des commentaires en ligne (`# ...`) et des commentaires de bloc (`{ ... }`), reconstruit le plateau à partir des symboles sérialisés, recharge l'historique en notation algébrique, puis vérifie la cohérence logique de l'état obtenu. Il peut aussi adapter la taille du contrôleur si la taille lue dans le fichier diffère de l'état courant. Enfin, la synchronisation réseau d'une sauvegarde chargée n'est prise en charge que dans les parties hôte-client ; elle n'est pas autorisée dans les parties client-client relayées.

L'implémentation sépare la sérialisation, la validation du contenu et la reconstruction de l'état. Charger une partie ne revient donc pas à lire un fichier : il faut aussi vérifier que le contenu décrit bien un état de jeu cohérent.

7.3.2 Intégration de la GUI

L'interface graphique GTK4 réutilise le même noyau métier que la CLI. Elle n'interprète pas elle-même les règles du jeu : elle transmet des intentions d'action au contrôleur, qui reste responsable de la validation, de l'application des coups et de la mise à jour de l'état. Ce choix limite les duplications de logique et facilite la cohérence entre les deux interfaces.

Sur le plan utilisateur, la GUI offre une lecture plus directe du plateau, des menus de configuration et des actions de sauvegarde ou de chargement. Les captures retenues dans le rapport ont été volontairement simplifiées : elles ne montrent que le plateau, l'indication du joueur courant et les marqueurs utiles au tour. L'icône de reine y représente une amazone ; le contour bleu désigne l'amazone actuellement sélectionnée ; les points noirs indiquent les cases légales de déplacement ou de tir selon la phase en cours. Cette convention rend la lecture des figures autonome, même pour un lecteur qui ne connaît pas encore l'interface.

Sur le plan logiciel, la GUI reste une vue branchée sur les mêmes objets métier. L'implémentation actuelle va toutefois au-delà d'un simple affichage local : `gui_config.py` porte la configuration réseau et le lobby hôte, `gui_menu.py` les actions de menus et de sauvegarde/chargement, et `gui_shortcuts.py` les raccourcis. La GUI gère aujourd'hui des matchs locaux, des matchs hôte contre client, un lobby multi-clients avec invitations, l'affichage du scoreboard, et le lancement d'une partie client contre client relayée par le serveur. Sa maturité reste toutefois inégale selon les scénarios, en particulier du côté de l'ergonomie et de la validation automatique.

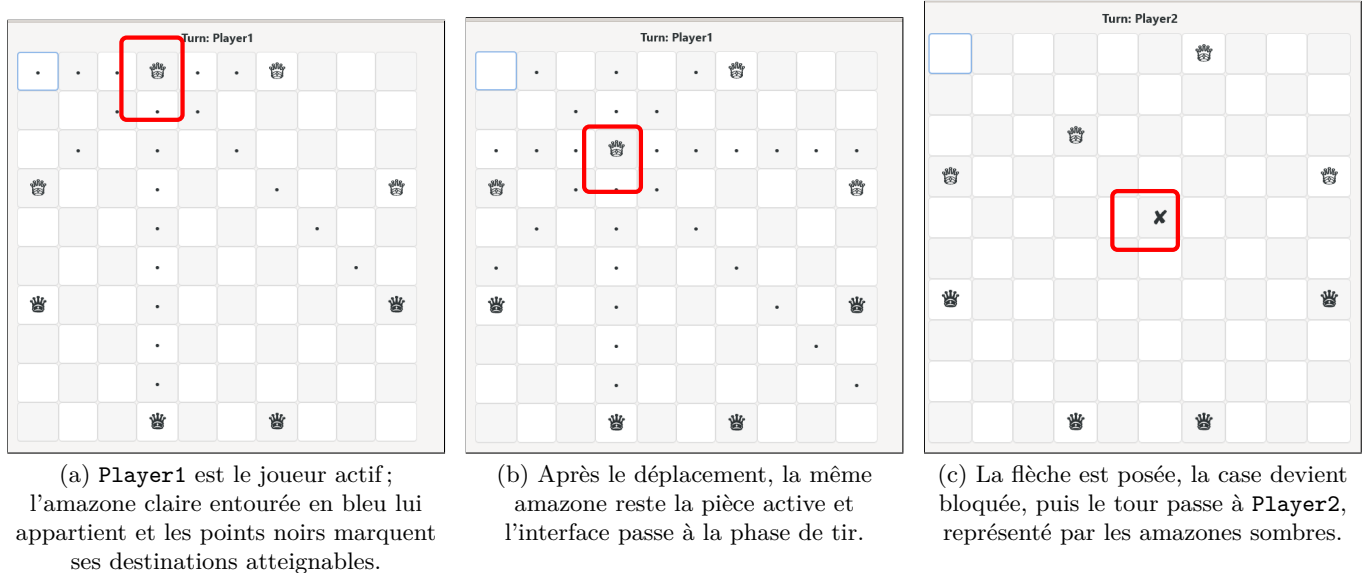


FIGURE 7.1 – Déroulement d'un tour dans l'interface graphique : sélection d'une amazone du joueur courant, déplacement, puis tir de flèche.

7.3.3 Intégration du réseau

Le réseau a été implémenté comme un service additionnel, et non comme une logique parallèle au moteur. Les messages reçus à distance sont convertis vers les mêmes objets et les mêmes opérations que dans une partie locale. Cette stratégie a deux avantages : elle limite les divergences entre modes de jeu et elle permet de conserver une unique source de vérité pour la validation des coups.

8 Tests et validation

La validation du projet combine tests automatisés, vérifications manuelles ciblées et essais en situation réelle. Cette approche est nécessaire : le moteur bitboard et le chargeur de sauvegardes se testent bien unitairement, alors que la GUI et certains scénarios réseau demandent encore des vérifications interactives. Le but est de vérifier que les invariants tiennent quand on combine moteur, interfaces, IA et persistance.

8.1 Stratégie de test

Le Tableau 8.1 résume le mode de validation privilégié pour chaque grand composant du projet. Il sert surtout à distinguer ce qui relève de tests automatisés stables et ce qui relève encore d'essais guidés.

Composant	Mode de validation privilégié
Moteur bitboard	tests unitaires sur la génération, l'application et l'annulation des coups
Persistance	scénarios de sauvegarde/chargement, validations de format et de checksum
CLI et contrôleur	tests d'intégration sur les commandes, l'historique et les transitions de tour
IA	scénarios de sélection de coup, évaluateurs et budgets temps
Réseau et GUI	tests manuels guidés et essais de cohérence fonctionnelle

TABLE 8.1 – Répartition des stratégies de validation selon les composants.

Le moteur bitboard est le composant le plus simple à valider automatiquement, car il expose des invariants nets : un coup légal doit être généré, appliqué puis annulé sans corrompre l'état. La persistance est validée de la même manière, en reconstruisant des parties sérialisées puis en vérifiant la cohérence logique de l'état restauré. Des scénarios typiques consistent par exemple à jouer un coup, l'annuler, puis vérifier que les trois bitboards retrouvent exactement leur état initial, ou à charger une sauvegarde contenant une coordonnée invalide et vérifier que le chargeur la rejette explicitement.

La CLI et le contrôleur nécessitent une approche plus intégrative. Les tests portent sur des transitions complètes : lecture d'une commande, validation, mise à jour du plateau, gestion de l'historique et retour utilisateur. Enfin, le réseau et la GUI restent les zones les plus coûteuses à automatiser complètement, car ils dépendent d'événements

asynchrones, de l'interface graphique et de la synchronisation de plusieurs acteurs ; ils sont donc encore validés en partie par des scénarios manuels guidés, comme la création d'un lobby, l'invitation d'un client, l'envoi d'un coup, une demande d'undo ou le chargement d'une sauvegarde en mode hôte-client.

8.2 Outils et critères de validation

Le projet utilise `pytest` [9] pour automatiser une partie importante des validations, complété par des vérifications manuelles ciblées sur les scénarios GUI et réseau. Au moment de la rédaction, la suite contient **470 tests réussis** et la couverture mesurée atteint **86 %** sur le paquet `amazons`. Trois niveaux de validation doivent être distingués.

Le premier niveau est la **validation fonctionnelle locale**. Il concerne le moteur, la génération des coups, l'application et l'annulation, les conversions de notation et la persistance. À ce niveau, la vraie question est simple : le système préserve-t-il les invariants du jeu ? C'est ici que les tests unitaires ont le plus de valeur.

Le deuxième niveau est la **validation d'intégration**. Elle concerne les interactions entre modules : par exemple, vérifier qu'une commande CLI déclenche bien une transition cohérente dans le contrôleur, qu'une sauvegarde reconstruite correspond à un état jouable, ou qu'une IA produit un coup exploitable par le moteur sans corrompre l'historique. Ce niveau est déterminant car le projet repose sur un noyau métier partagé entre plusieurs points d'entrée.

Le troisième niveau est la **validation système**. Elle cible les scénarios où plusieurs couches interagissent simultanément : partie réseau, chronométrage, invitations, GUI et synchronisation d'état. C'est également le niveau le plus difficile à automatiser complètement, car il dépend d'événements asynchrones et d'interactions utilisateur.

Les chiffres de tests doivent donc être lus avec prudence. Un grand nombre de tests réussis rassure sur le noyau, mais ne garantit pas à lui seul la stabilité de toutes les interactions entre interfaces. La validation doit être jugée sur les risques réellement couverts, pas sur le nombre de tests seul.

9 Analyse de performances

Cette section relie quelques mesures simples aux choix algorithmiques du projet. L'objectif n'est pas de produire un benchmark industriel, mais d'illustrer la croissance du facteur de branchement, la sensibilité de Minimax à la profondeur et le comportement *anytime* de MCTS.

9.1 Mesures

Taille	Coups initiaux	Minimax $d = 1$	Minimax $d = 2$	MCTS 0,5s
4×4	104	0,013 s	0,820 s	0,500 s
5×5	198	0,045 s	7,083 s	0,501 s
6×6	496	0,242 s	> 20 s	0,502 s
8×8	1232	1,359 s	> 20 s	0,503 s
10×10	2176	4,218 s	> 20 s	0,510 s

TABLE 9.1 – Mesures sur la position initiale : croissance du facteur de branchement et temps de décision.

Le premier résultat marquant est l'augmentation rapide du nombre de coups légaux. On passe de 104 coups sur un plateau 4×4 à 2176 coups sur un plateau 10×10 . Cette croissance explique directement pourquoi les recherches adversariales profondes deviennent vite difficiles à maintenir interactives.

Le deuxième résultat concerne Minimax. À profondeur 1, le moteur reste utilisable sur les petits plateaux, mais dépasse déjà 4 secondes sur la position initiale standard. À profondeur 2, le temps de calcul explose : le plateau 5×5 dépasse 7 secondes et les tailles supérieures dépassent 20 secondes. Cette évolution illustre concrètement la complexité en $O(b^d)$: dès que le facteur de branchement b devient grand, une faible augmentation de profondeur rend la recherche non interactive. La raison est simple : à chaque nœud, Minimax doit régénérer de très nombreux coups, explorer leurs successeurs et répéter ce travail récursivement.

Le troisième résultat concerne MCTS. Avec un budget de 0,5 seconde, le temps de réponse reste stable quelle que soit la taille du plateau. Ce comportement ne signifie pas que le problème devient facile ; il montre que le moteur respecte d'abord une contrainte de temps, puis améliore sa décision à l'intérieur de ce budget. MCTS utilise donc mieux le temps disponible, car sa boucle s'arrête naturellement à l'échéance au lieu de viser une profondeur fixe qui devient vite hors de portée.

9.2 Comparaison expérimentale des IA

Pour compléter les mesures précédentes, nous avons exécuté le script `scripts/benchmark_ai.py` sur deux confrontations courtes. Le script alterne les couleurs d'une partie à l'autre, ce qui évite de confondre l'effet du moteur avec l'avantage éventuel du trait. La Figure 9.1 montre les deux sorties retenues : `minimax` contre `mcts` sur plateau 6×6 , puis `minimax` contre `random` sur plateau 10×10 .

(a) Minimax vs MCTS sur 6×6

```

PS C:\Users\pc\Desktop\POP\amazons-python> python scripts/benchmark_ai.py --p1 minimax --p2 mcts --games 6 --size 6
=== BENCHMARK: minimax vs mcts ===
Parties: 6 | Plateau: 6x6 | Temps Global: 10.0s | Temps/Coup: 5.0s | Profondeur: 2 | Threads: 12
Lancement des matchs en cours...

Progression : 6/6

=== RÉSULTATS ===

Match: minimax vs mcts (6 parties)
-----
Victoires minimax: 3 (50.0%)
Victoires mcts: 3 (50.0%)
Égalités/Erreurs: 0
-----
Raisons de victoire :
- temps écoulé: 6
-----
Moyenne de coups par partie : 2.0
Temps CPU total (somme de tous les cours) : 53.38s
Temps réel écoulé : 16.54s (Accélération x1.2)

```

(b) Minimax vs Random sur 10×10

```

PS C:\Users\pc\Desktop\POP\amazons-python> python scripts/benchmark_ai.py --p1 minimax --p2 random --games 6 --size 10 --workers 4
=== BENCHMARK: minimax vs random ===
Parties: 6 | Plateau: 10x10 | Temps Global: 10.0s | Temps/Coup: 5.0s | Profondeur: 2 | Threads: 4
Lancement des matchs en cours...

Progression : 6/6

=== RÉSULTATS ===

Match: minimax vs random (6 parties)
-----
Victoires minimax: 0 (0.0%)
Victoires random: 6 (100.0%)
Égalités/Erreurs: 0
-----
Raisons de victoire :
- temps écoulé: 6
-----
Moyenne de coups par partie : 0.5
Temps CPU total (somme de tous les cours) : 38.66s
Temps réel écoulé : 29.07s (Accélération x1.3)

```

FIGURE 9.1 – Captures du benchmark expérimental des IA : 6 parties, couleurs alternées, profondeur Minimax 2, budget de 5 secondes par coup et 10 secondes par joueur.

Le premier résultat est un équilibre apparent : sur six parties, **minimax** et **mcts** gagnent chacun trois fois. Ce 50/50 ne prouve pas que les deux moteurs se valent en général. Il reflète surtout ce réglage précis : un échantillon très court, un plateau 6×6 où la profondeur 2 reste encore jouable pour Minimax, et un budget temps trop faible pour que MCTS stabilise vraiment ses statistiques. Dans ces conditions, aucun moteur ne prend un avantage net.

Le comportement des deux IA reste pourtant différent. Minimax est déterministe : à plateau et paramètres identiques, il parcourt les coups dans le même ordre et applique la même heuristique. Ses décisions sont stables, mais leur qualité dépend directement de la profondeur atteinte. MCTS est plus exploratoire : la sélection suit UCT, puis la simulation repose sur des coups aléatoires. Il peut trouver de bons coups sans profondeur fixe, mais il reste moins stable d'une partie à l'autre.

Le second résultat est plus utile que surprenant : **random** gagne les six parties face à **minimax** sur plateau 10×10 , et les six victoires sont comptées pour « temps écoulé ». Le benchmark ne dit donc pas que **random** joue mieux. Il montre qu'avec profondeur 2 et 5 secondes par coup, Minimax dépasse le budget sur grand plateau, alors que **random** répond presque immédiatement. Dans cette expérience, on mesure d'abord la tenue d'un moteur sous contrainte de temps.

Au vu de ces essais, Minimax reste adapté aux petits et moyens plateaux, ou aux cas où l'on veut une décision stable et explicable. MCTS devient plus crédible dès que le facteur de branchement monte et qu'il faut surtout garantir une réponse dans le temps imparti. Le benchmark ne désigne donc pas un vainqueur universel ; il montre plutôt dans quels cas chaque moteur reste utilisable.

9.3 Interprétation

Ces mesures doivent être replacées dans le cadre théorique du jeu. Les Amazones sont connus comme **PSPACE-complets** [4]. Il n'est donc pas réaliste d'attendre un solveur exact efficace sur les positions générales. Les bitboards ne suppriment pas cette difficulté ; ils évitent surtout que la représentation mémoire devienne le premier goulet d'étranglement.

La représentation bitboard permet de maintenir une empreinte mémoire compacte : l'état du plateau reste concentré dans trois entiers Python, ce qui réduit le coût des mises à jour locales et des copies logiques. Le comportement mémoire diffère ensuite selon le moteur. Le Minimax avec **make_move/undo_move** consomme surtout du temps CPU. Le MCTS déplace une partie du coût vers l'arbre de recherche, puisque chaque nœud stocke des statistiques, des enfants et des coups non encore essayés. Les bitboards ne résolvent donc pas la complexité du jeu ; ils évitent surtout que la représentation interne devienne elle-même le premier goulet d'étranglement.

10 Discussion critique

Cette section revient sur les limites réellement rencontrées pendant le développement et la validation. Elle ne reprend pas l'annexe des besoins point par point ; elle met l'accent sur les fragilités techniques du système, les compromis retenus et les améliorations encore utiles.

10.1 Limites algorithmiques

- **Minimax reste fortement limité par la profondeur.** Même avec l'élagage $\alpha\beta$, l'explosion combinatoire apparaît très vite dès que le plateau s'ouvre.
- **MCTS dépend de la qualité des rollouts.** Les simulations restent volontairement simples dans cette version ; elles sont robustes pour un prototype académique, mais insuffisantes pour atteindre une forte qualité stratégique.
- **Les heuristiques restent simples.** L'évaluateur territorial repose sur une propagation locale rapide, et l'évaluateur hybride conserve des poids fixes. Ce choix favorise la lisibilité et le temps de calcul, mais limite la

finesse de l'évaluation.

10.2 Limites architecturales

- **Le contrôleur reste trop central.** `GameController` conserve une part trop importante de l'orchestration applicative : boucle de jeu, historique, persistance, temps et réseau.
- **La factorisation reste partielle.** Les modules de `controller/engine` améliorent la lisibilité locale, mais ils restent fortement couplés au même état contrôleur.
- **La frontière vue / contrôleur est correcte mais encore lourde.** La CLI et la GUI réutilisent bien le même noyau métier, mais la GUI porte encore beaucoup de logique d'orchestration événementielle.
- **Certaines incohérences internes subsistent.** Le refactoring du contrôleur est réel, mais il n'est pas totalement stabilisé : certaines responsabilités restent dupliquées et certaines commandes avancées sont mieux supportées par le contrôleur que par l'aide CLI.

10.3 Limites système

- **Le protocole réseau reste fragile.** Son format textuel est pratique pour le débogage, mais moins robuste qu'un protocole typé ou sérialisé.
- **La validation automatique n'est pas homogène.** Le moteur, la persistance et une grande partie du contrôleur sont bien couverts ; la GUI et certains scénarios réseau restent plus dépendants de tests manuels.
- **Certaines fonctionnalités restent incomplètes.** L'option `-ai-mcts-selection ML` est visible, mais elle retombe encore sur UCT avec un avertissement ; le *drag and drop* GUI reste partiel selon les phases du tour.
- **Tout n'est pas industrialisé.** Certains assets de diagrammes ou de génération restent encore dépendants d'une manipulation manuelle plutôt que d'une chaîne de production entièrement automatisée.

Ces limites ne remettent pas en cause le fonctionnement du moteur. Elles montrent surtout que le projet a privilégié la convergence fonctionnelle et la cohérence globale du système, avant l'industrialisation complète de tous les sous-modules.

10.4 Améliorations possibles

Les améliorations futures les plus crédibles peuvent être ordonnées selon trois priorités. La première est **algorithmique** : ajout de tables de transposition, ordonnancement des coups, rollouts MCTS guidés et, à plus long terme, apprentissage d'une fonction d'évaluation. La deuxième est **architecturale** : extraction de `GameController` en services applicatifs plus petits pour mieux séparer orchestration de partie, réseau, persistance et logique d'interface. La troisième est **système** : protocole réseau plus structuré, tests GUI/réseau plus poussés et instrumentation de performance plus systématique.

11 Apports du projet

Le projet a réuni dans un même système des choix algorithmiques, des décisions d'architecture et des exigences de validation.

11.1 Apports techniques

Les apports techniques essentiels sont au nombre de trois : une représentation bitboard adaptée aux opérations critiques du moteur, plusieurs moteurs d'IA comparables sous une même interface, et un noyau métier partagé entre CLI, GUI, persistance et réseau.

11.2 Apports méthodologiques

Le projet a surtout renforcé des compétences de découpage modulaire, de validation progressive et de refactorisation sous contrainte. Il a aussi rendu visibles plusieurs compromis d'ingénierie : plus de découpage améliore la maintenabilité, mais augmente le coût de stabilisation ; une IA plus ambitieuse peut améliorer le jeu, mais complique l'évaluation et les tests.

11.3 Vision d'ingénieur

Le principal enseignement du projet est simple : une solution utile doit rester compréhensible, testable et améliorable.

Conclusion

Ce projet montre qu'un jeu comme les Amazones pose plusieurs difficultés en même temps. La première est algorithmique : facteur de branchement élevé, besoin d'heuristiques et arbitrage entre recherche déterministe et estimation statistique. La deuxième est logicielle : représentation efficace de l'état, préservation des invariants, persistance, réseau et pluralité des interfaces. La troisième est méthodologique : justifier des choix imparfaits mais cohérents avec un temps de développement limité.

Les contributions les plus nettes sont les suivantes : un noyau métier réutilisable, une représentation bitboard adaptée aux opérations critiques, plusieurs moteurs d'IA comparables sous une même abstraction, et un système complet combinant CLI, GUI, réseau, sauvegarde et internationalisation.

Le travail met aussi en évidence des compromis clairs : une architecture assez modulaire mais encore trop centralisée autour du contrôleur, des heuristiques simples pour garder un temps de réponse interactif, et un protocole réseau léger pour rester débogable. Le projet fournit ainsi une base utile pour améliorer le moteur, poursuivre la factorisation architecturale, renforcer le protocole réseau et introduire des techniques d'apprentissage.

Références

- [1] Cameron Browne. Bitboard methods for games. *ICGA Journal*, 37(2) :67–84, June 2014.
- [2] Cameron Browne, Edward Powley, Daniel Whitehouse, Simon Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1) :1–43, 2012.
- [3] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns : Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [4] Robert A. Hearn. Amazons is PSPACE-complete. *Computing Research Repository (CoRR)*, abs/cs/0502013, 2005.
- [5] Donald E. Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4) :293–326, 1975.
- [6] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *Machine Learning : ECML 2006*, volume 4212 of *Lecture Notes in Computer Science*, pages 282–293, Berlin, Heidelberg, 2006. Springer.
- [7] Michael Lieberum. An evaluation function for the game of amazons. In *Advances in Computer Games 9*, pages 49–56. Universitaire Pers Maastricht, 1999.
- [8] Martin Müller and Theodore Tegos. Experiments in computer amazons. In *MSRI Publications*, volume 42, pages 243–260. Cambridge University Press, 2002.
- [9] pytest Development Team. *pytest Documentation*. pytest Development Team, 2024. Accessed : 2026-01-29.
- [10] Python Software Foundation. *gettext — Multilingual internationalization services*. Python Software Foundation, 2024. Accessed : 2026-01-29.
- [11] Python Software Foundation. *socket — Low-level networking interface*. Python Software Foundation, 2024. Accessed : 2026-01-29.

A Liste des 40 besoins fonctionnels

ID	Description	Statut
F1	Options de lancement en ligne de commande	Fait
F2	Configuration utilisateur (taille, mode, temps)	Fait
F3	Internationalisation (anglais/français)	Fait
F4	Mode CLI interactif	Fait
F5	Mode Blitz (chronomètre)	Fait
F6	Mode Contest (tournoi)	Fait
F7	Interface graphique GTK4	Partiel
F8	Jouer contre l'IA	Fait
F9	Taille de plateau variable (4–11)	Fait
F10	Détection de fin de partie	Fait
F11	Validation des coups et erreurs	Fait
F12	Undo/Redo avec accord adverse	Fait
F13	Undo/Redo de N coups	Fait
F14	Affichage de l'historique des coups	Fait
F15	Shell interactif avec prompt	Fait
F16	Complétion automatique (tab)	Fait
F17	Aide contextuelle	Fait
F18	Affichage du plateau en ASCII	Fait
F19	Affichage coloré du plateau	Non fait
F20	Notation algébrique	Fait
F21	Sauvegarde au format ASCII	Fait
F22	Sauvegarde des paramètres de jeu	Fait
F23	Sauvegarde de l'état du plateau	Fait
F24	Sauvegarde de l'historique	Fait
F25	Chargement d'une partie sauvegardée	Fait
F26	Représentation bitboard	Fait
F27	Algorithmes bitboard (5 opérations)	Fait
F28	Menus graphiques	Fait
F29	Raccourcis clavier GUI	Fait
F30	Drag & Drop GUI	Partiel
F31	Minimax	Fait
F32	Élagage alpha-bêta	Fait
F33	Évaluation par mobilité	Fait
F34	Évaluation par territoire	Fait
F35	MCTS	Fait
F36	Évaluation hybride	Fait
F37	Machine Learning (Keras)	Non fait
F38	Découverte serveur UDP	Fait
F39	Jeu en réseau TCP	Fait
F40	Système d'invitations	Fait

Bilan : 36 besoins réalisés, 2 partiels, 2 non réalisés (F19, F37).

Précisions sur les statuts non complets.

- **F7 – Interface graphique GTK4 :** statut partiel car la GUI couvre les scénarios principaux, mais certaines interactions restent moins abouties que dans la CLI et moins bien validées automatiquement.
- **F30 – Drag & Drop GUI :** statut partiel car le geste est présent dans l'interface, mais son comportement n'est pas encore homogène dans tous les scénarios et toutes les phases du tour.
- **F19 – Affichage coloré du plateau :** non réalisé dans la CLI finale; le plateau reste lisible, mais sans enrichissement couleur.
- **F37 – Machine Learning (Keras) :** non réalisé; l'option a été laissée comme piste d'extension, sans moteur d'apprentissage effectivement branché.